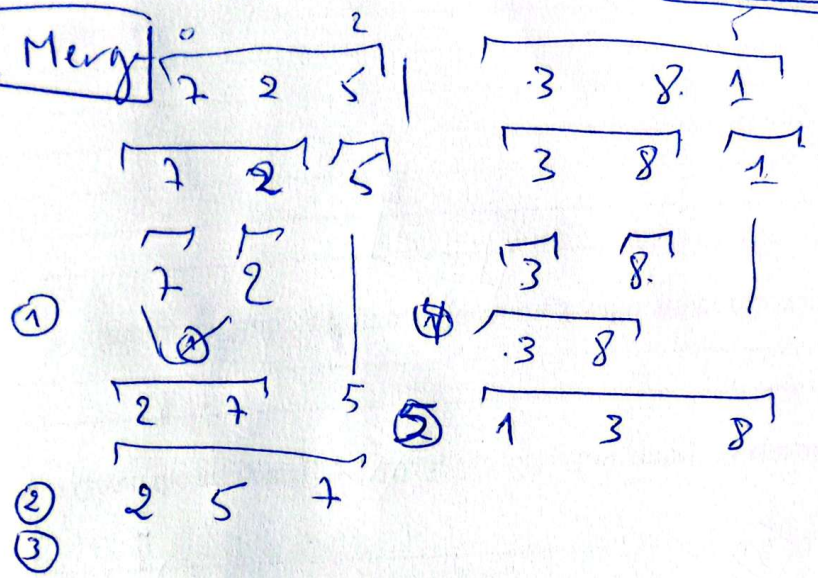
Exercice 1 — 7,5

Page #1

① 7 2 5 3 8 1

Selection 5 + 4 + 3 + 2 + 1 = 15 comparisons

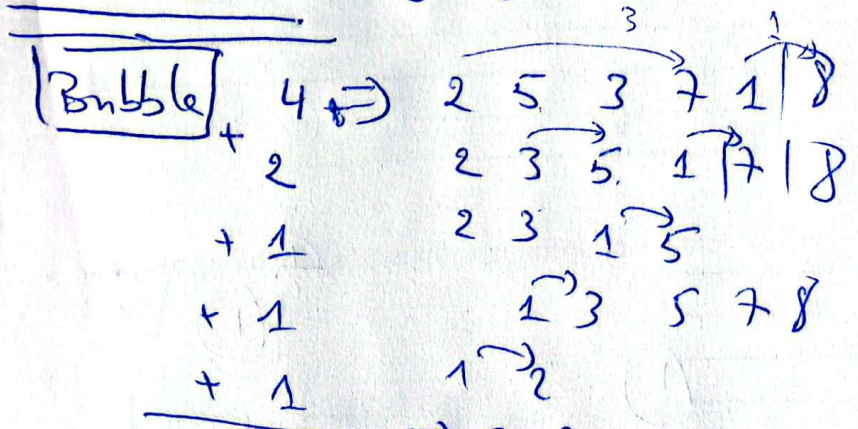
Merge



1, r

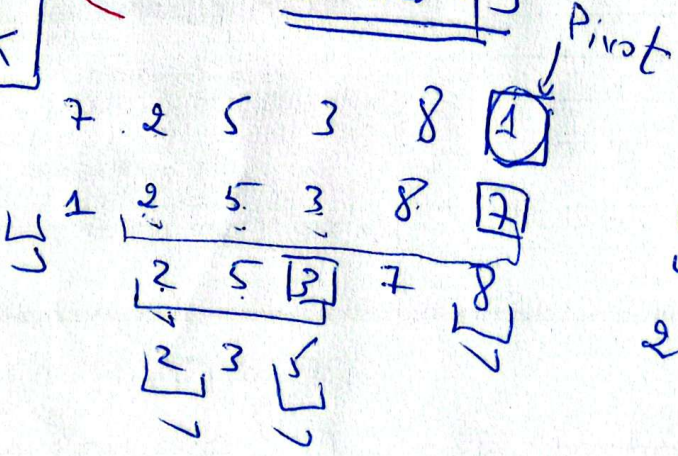
10 comparisons

Bubble



⇒ 9 swaps

Quick



1, r

① swap
4 swaps
2 swaps

7 Swaps

(2) (1) int duplicate (int A[], int N){

for (int i=0; i<n-1; i++)

(1) for (int j = i+1; j<n; j++)

if (A[i]== A[j]) return 1;

return 0; }

Complexity: worst = $\frac{n(n-1)}{2} = O(n^2)$ (9,r)

(2,5) int duplicate2 (int A[], int N){

(1,r) Sort (A, N); // use any sorting method with n log n
// or less

(1) for (int i=0; i<n; i++)

if (A[i]== A[i+1]) return 1;

return 0;

}

Complexity: worst = Complexity (sort) + Complexity (for)

(3) = n log n + n

= n log n

⇒ So, Sorting step improves efficiency

(0,r)

Exercise 2 — 3

1) `int exist (NgPtr root, int id){`

0,5 `if ( ! root) return 0;`

0,5 `if (root->data == id) return 1;`

2 `return exist(root->Child) || exist(root->next); }`

2) — 5,5

`int Cousin (NgPtr root, int id){`

0,5 `if(root == NULL) return -1; // Not found`

0,5 `NgPtr f = father (root, id);` — 2

0,5 `if ( f == NULL) return -1; // Not found`

```
int Count=0;
NgPtr uncle = f->next;
1 while (uncle) { NgPtr c = uncle->Child;
                  1 while (c) { Count++;
                                c = c->next; }
                  uncle = uncle->next; }
return Count; }
```

2

⇒ `NgPtr father( NgPtr root, int id) {`

0,5 `if(!root) return NULL;`

`NgPtr c = root->Child;`

0,25 `while (c) { if (c->data == id) return root; c = c->next; }`

0,5 `NgPtr r = father (root->Child, id); if (r) return r;`

0,5 `return father (root->next, id); }`

Exercice 3 (4)

Global

```
int value = 1;  or
```

```
NPtr create (int level){
or if (level == 0) return NULL;
(1) NPtr Lchild = create(level-1);
(1) Rchild = create(level-1);
    NPtr new = (NPtr) malloc(sizeof(ND));
(1)     new->info = value++;
        new->left = Lchild;
        new->right = Rchild;
    return new; }
```